

Algorithm Performance, Computational Complexity

ITCS 4114: Real World Algorithms

Kalpathi Subramanian

Computer Science, UNC Charlotte

Last Modified: January 2026

How Can We Measure Algorithm Performance

Empirical comparisons (run programs) has difficulties

- ◇ Programming effort
- ◇ Implementation differences
- ◇ Exhaustive testing infeasible
- ◇ At best, want to keep 1 implementation

Alternately, Asymptotic Algorithm Analysis

- ◇ Measure performance as a function of input size.
- ◇ Can be **unrealistic**
 - ▷ Today's computer architectures are very sophisticated and support parallelism.
 - ▷ GPUs are very common, CPUs have multiple cores
 - ▷ Hard to match theoretical complexity functions

Algorithm Performance: What do we care about?

Resources

- ◇ Running time.
- ◇ Space (RAM, Disk).
- ◇ Programmer's effort.
- ◇ Ease of use (user's effort).

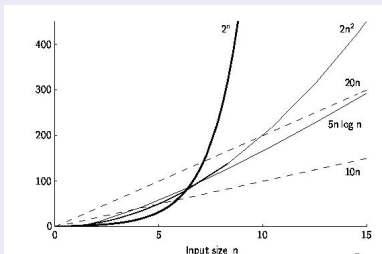
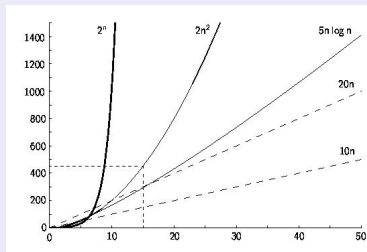
Performance depends on

- ◇ Machine load
- ◇ OS, Compiler
- ◇ Problem size
- ◇ Input values

For most algorithms, running time, $T(n)$ depends on some function of "size" n , of the input.

$$T(n) \propto f(n)$$

Growth Rate Functions



- ◇ Plots show **size of input** (X axis) vs. **Resources** (compute time, memory, etc))
- ◇ **Increasing order of complexity:** Constant, Linear, $\log n$, $n \log n$, n^2 (quadratic), 2^n (exponential), $n!$
- ◇ **Constants** play a role on where the various functions cross for smaller values of input size

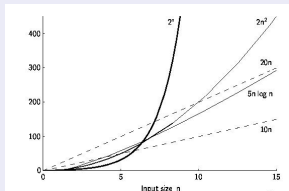
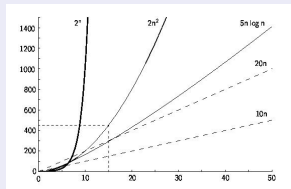
Input Size vs. Computation Speedup

T_n	n	n'	Change	n'/n
$10n$	1,000	10,000	$n' = 10n$	10
$20n$	500	5,000	$n' = 10n$	10
$5n \lg n$	250	1,842	$\sqrt{10}n < n' < 10n$	7.37
$2n^2$	70	223	$n' = \sqrt{10}n$	3.16
2^n	13	16	$n' = n + 3$	—

- ◇ Comparing input size that can be computed by a system that is 10 times more powerful, given the algorithm growth rate (10,000 ops/hour vs. 100,000 ops/hour).
- ◇ As the complexity increases, the size of input processed by the more powerful system drops.

Asymptotic Analysis

Asymptotic analysis is defined as the study algorithm performance and comparison as the input size n gets big or reaches a limit

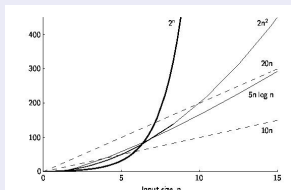
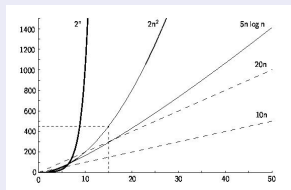


- ◇ Curves $2n^2$ and $10n$ cross at $n = 5$
- ◇ Curves $2n^2$ surpasses $20n$ cross at about $n = 10$
- ◇ Constants only **shift the crossover point**
- ◇ For asymptotic analyses, we **ignore the constants**
- ◇ What is important are the **growth rates** of the algorithm, as shown above

Asymptotic analysis can be thought of as a 'back of the envelope' calculation for estimating algorithm resource consumption, providing a simplified model of algorithm running time, memory or other resource needs

Asymptotic Analysis: Upper Bounds

Upper bound for an algorithm is the upper or highest growth rate that the algorithm can have



- ◇ Upper bound is **not the worst case** of an algorithm!
- ◇ Worst case is specific to a set of inputs for an algorithm performance.
- ◇ Upper bounds are measured for a **class of inputs** of size n , typically for its best, average or worst case
- ◇ For instance, you can state *'this algorithm has an upper bound to its growth rate of n^2 in the average case'*

Asymptotic Analysis (or Computational Complexity)

Provides a formal way to measure performance of algorithms

Big Oh Bounds

T_n is in the set $O(f(n))$ if there exist two positive constants c and n_0 such that $|T(n)| \leq c|f(n)|$, for all $n > n_0$.

- ◇ For **all** data sets (i.e., $n > n_0$), the algorithm **always** executes in less than $c|f(n)|$ steps (in best, average or worst case).
- ◇ Big-oh provides an **Upper Bound**
- ◇ Prefer the **tightest** upper bound: $T_n = 3n^2$ is in $O(n^3)$, but prefer $O(n^2)$.

Example: $T_n = c_s n/2$ (sequential search)

For all values of $n > 1$,

$$c_s \cdot n/2 \leq c_s \cdot n$$

Therefore, by definition, T_n is in $O(n)$ for $n_0 = 1$, $c = c_s$.

Big Ω Bound

T_n is in the set $\Omega(g(n))$ if there exist two positive constants c and n_0 such that $T_n \geq cg(n)$ for all $n > n_0$.

- ◇ For **all** data sets big enough (i.e., $n > n_0$), the algorithm **always** executes in more than $cg(n)$ steps.
- ◇ Big Omega provides a **lower bound**.
- ◇ Always want greatest lower bound.

Example: $T_n = c_1n^2 + c_2n$

$$\begin{aligned} T_n &= c_1n^2 + c_2n \\ c_1n^2 + c_2n &\geq c_1n^2, \forall n > 1 \\ T_n &\geq cn^2, \text{ for } c = c_1, n_0 = 1. \end{aligned}$$

Therefore, T_n is in $\Omega(n^2)$, by definition.

Big Θ Bound

An algorithm is said to be $\Theta(h(n))$ if it is in $O(h(n))$ and it is in $\Omega(h(n))$.

- ◇ For polynomial equations on T_n , we always have Θ bounds.

Simplification Rules

- ◇ If $f(n)$ is in $O(g(n))$ and $g(n)$ is in $O(h(n))$, then $f(n)$ is in $O(h(n))$
- ◇ If $f(n)$ is in $O(kg(n))$ for any constant $k > 0$, then $f(n)$ is in $O(g(n))$
- ◇ If $f_1(n)$ is in $O(g_1(n))$ and $f_2(n)$ is in $O(g_2(n))$, then $(f_1 + f_2)(n)$ is in $O(\max(g_1(n), g_2(n)))$.
- ◇ If $f_1(n)$ is in $O(g_1(n))$ and $f_2(n)$ is in $O(g_2(n))$ then $f_1(n)f_2(n)$ is in $O(g_1(n)g_2(n))$.

Space Complexity

- ◇ **Time Complexity** is generally relevant for **algorithms**
- ◇ **Space Complexity** is generally relevant for **data structures**
- ◇ **Space/Time Tradeoff**: Reduction in computation time by sacrificing space, or vice versa; **Examples**: table look up (indexing), search trees
- ◇ **Disk Based Space/Time Tradeoff**: Smaller disk storage requirements, the faster the algorithm

Selection Sort Running Time vs Input Size

Experimental Runs

- ◇ A run with size $N = 400,000$ (1.6M storage) takes about 170s

N	Bytes	Time	Remarks
400,000	1.6M	168sec	Floppy Disc
175 Million	700M	1.05 years	CD
1.1 G	4.4G	34 years	DVD
5G	20G	859 years	Blue Ray

Complexity Problems

Use the standard definitions of Big Oh, Big Ω , derive the complexity of the following:

- ◇ $n(n+1)/2 \in O(n^3)$
- ◇ $n(n+1)/2 \in O(n^2)$
- ◇ $n(n+1)/2 \in \Omega(n)$
- ◇ $c_4 n \log n + c_5 n \in \Omega(n \log n)$
- ◇ $c_6 2^n + c_7 n^6 \in \Omega(2^n)$
- ◇ $5 + 0.001n^3 + 0.025n \in \Omega(n^3)$
- ◇ $n^2 \log n + n(\log n)^2 \in O(n^2 \log n)$
- ◇ $100n + 0.01n^2 \in O(n^2)$

Complexity Problems

Determine the Big Oh complexity of the following program fragments:

```
-----  
void Sum0(int n) {  
    sum1 = 0  
    for (i=0; i<3; i++) //First loop  
        for (j=0; j<n; j++) // do n times  
            sum++;  
}
```

```
-----  
void Sum1(int n){  
    sum1 = 0;  
    for (i=1; i<=n; i++) //First loop  
        for (j=1; j<=n; j++) // do n times  
            sum1++;  
}
```

```
-----  
void Sum2(int n) {  
    sum2 = 0;  
    for (i=1; i<=n; i++) // first loop n times  
        for (j=1; j<=i; j++) // do i times  
            sum2++;  
}
```

```
-----  
void randops (int n) {  
    for (i=1; i<n; i++) {  
        for (j=1; j<n; j++)  
            A[i] = random(n); // random takes constant time  
        sort(A, n);           // sort takes n log n  
    }  
}
```

Complexity Problems

```
-----  
float useless(A){  
    n = A.length;  
    if (n==1){  
        return A[0];  
    }  
    // let A1,A2 be arrays of size n/2  
    for (i=0; i <= (n/2)-1; i++){  
        A1[i] = A[i];  
        A2[i] = A[n/2 + i];  
    }  
    for (i=0; i<=(n/2)-1; i++){  
        for (j=i+1; j<=(n/2)-1; j++){  
            if (A1[i] == A2[j])  
                A2[j] = 0;  
        }  
    }  
    b1 = useless(A1);  
    b2 = useless(A2);  
    return max(b1,b2);  
}
```